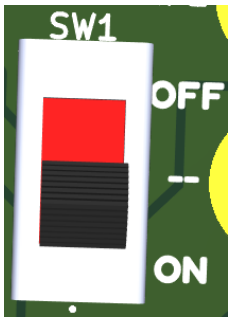
Agricultural VCU V2 Testing Protocol

- Soldering tips
  - Top layer first, then bottom. All SMD components must be soldered before soldering through-hole components
  - (Recommended) Soldering steps:
  1. Power supply
  2. MCU circuit (decoupling capacitors and crystal)
  3. GPIO-related SMD components
  4. Ethernet
  - When soldering capacitors, always check the connectivity after each capacitor is soldered. A capacitor is quite sensitive to temperature limits; when it is heated for too long, it tends to "kill" itself, resulting in a short circuit. Short-circuit capacitors affect the PCB performance.
- Power

  12V: Using DC Power Supply, apply 12V to the voltage input terminal block (J1). Set the switch to ON and check the voltage at TP1.

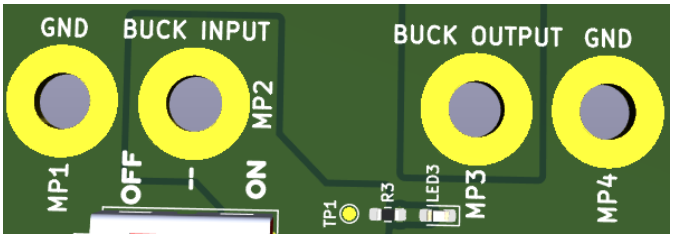
  Ensure the black tab of the switch is set to the ON side, not in the middle.



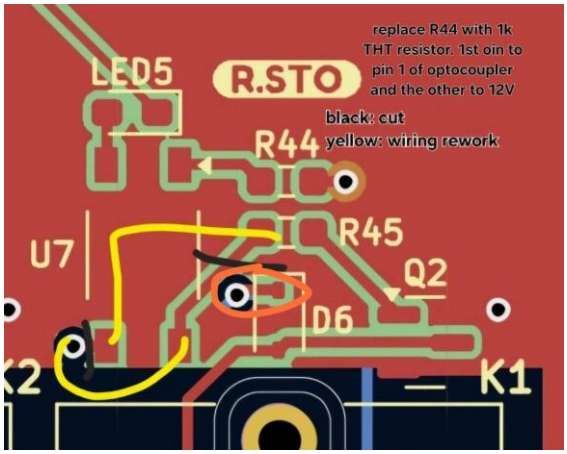
  Before installing the buck module, attach a round or U crimp (round is preferred) cable connector with a cable crimper to each buck module cable.



  5V and 3V3: Install each buck module cable on the spacers using an M3 screw. The spacer's position order is seen below



  Check the voltage at TP2 for the 5V rail and at TP3 for the 3V3 rail. The 3V3 is obtained from voltage regulator TPS79533 output. There is also an LED indicator for each rail, but measuring the voltage with a multimeter on test point is more accurate. Also, check on various points for both voltage rails.
- Rework tutorial for relay circuit



1. Power off the PCB completely and disconnect all external supplies, debugger cables, and relay wiring.
2. Identify the target relay channel and locate R44, the corresponding optocoupler, and the nearby 12 V routing point shown in the layout.
3. Remove the original R44 component from the PCB.
4. Prepare a 1 kΩ THT resistor for the rework.
5. Cut the original PCB trace at the location marked in black to isolate the previous low-voltage pull-up path.
6. Verify with a multimeter that the cut trace is open and no longer electrically continuous.
7. Solder one lead of the 1 kΩ THT resistor to pin 1 of the optocoupler.
8. Route the other lead of the 1 kΩ THT resistor to the 12 V rail
9. Use the pad/point marked by the orange circle as an alternative solder point for the 12 V wiring rework if it is mechanically easier or gives a shorter connection.
10. Ensure the rework wire does not short to adjacent pads, vias, or traces, and secure the wire so it does not move during later handling.
11. Inspect the reworked area visually for solder bridges, insufficient wetting, or exposed conductor touching nearby copper.
12. Measure continuity to confirm: optocoupler pin 1 is connected to 12 V through the new 1 kΩ resistor, the old pull-up path is disconnected, and no short circuit exists between 12 V, GND, and adjacent signal nets.
13. Repeat the same rework procedure for the other relay channels that use the same component placement and trace position.
14. After completing all relay-channel reworks, perform a final continuity check on every modified channel before powering the PCB again.

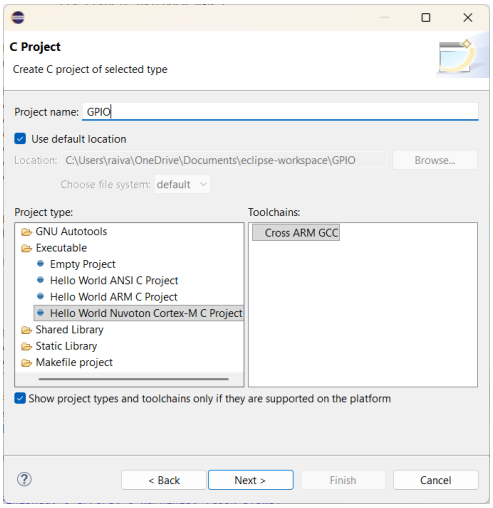
- Debugging
  1. Connect NuLink-Pro to SWD header (J3). Right-angle mark at the side of the connector marks the first pin of the pin header. For NuLink-Pro, the pinout is shown below



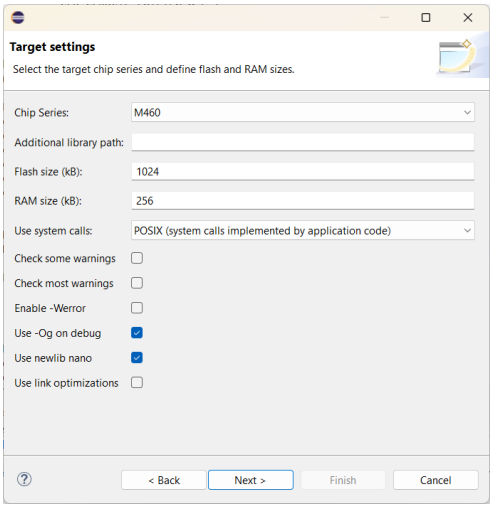
Wiring between NuLink-Pro and SWD header:

NuLink-Pro	SWD pin header
VCC	1
ICE_DAT	2
ICE_CLK	3
/(RESET)	4
VSS	5

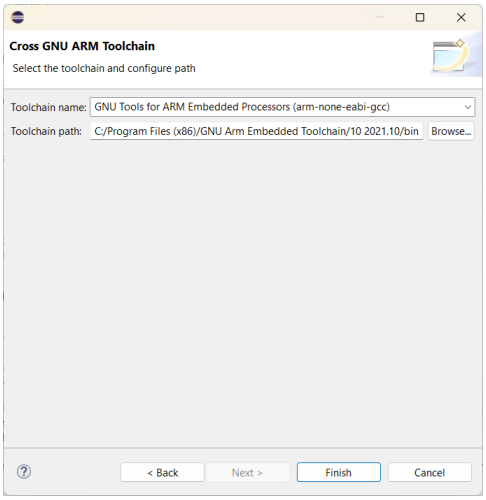
2. Create a project:
   a. Click File → New → C/C++ Project
   b. In All section, choose C Managed Build
   c. Write the project name. On “Project Type” choose Executable→Hello World Nuvoton Cortex-M C Project



   d. Target settings:
      Chip Series: M460
      Keep other settings as default



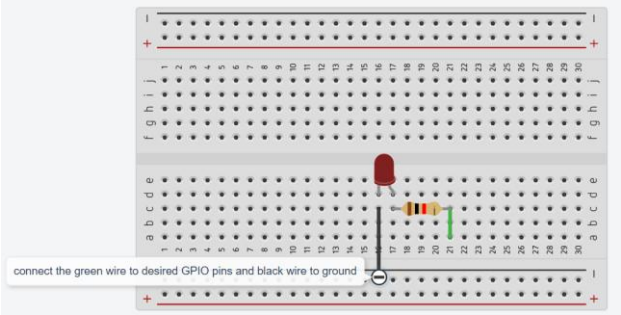
e. Keep the “Select Configurations” option as default and click next
f. Keep the toolchain as default and click finish



3. Check the Library/StdDriver folder. If there is no src folder inside, manually add a folder named src and copy necessary .c library files from Nuvoton M460 board support package (BSP) in Library/StdDriver/src folder (link:https://github.com/OpenNuvoton/m460bsp/tree/master/Library/StdDriver/src)
4. Write a code in User/main.c and save the file
5. Build (click the hammer symbol in the IDE) and debug () to execute the program
   a. On debug icon, click the arrow and select “Debug Configurations”
   b. Double click on “GDB Nuvoton Nu-Link Debugging”
   c. Keep the default settings after the debugging cconfiguration shown.
   d. Click debug

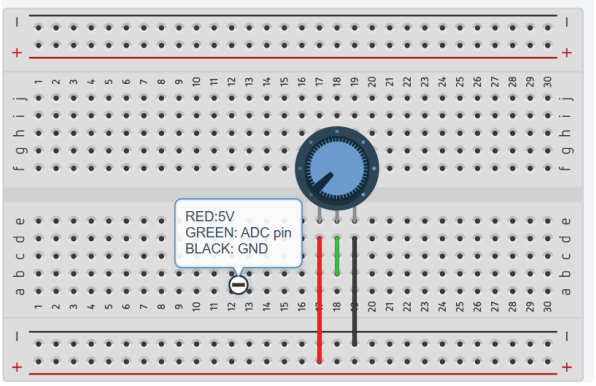
To test the functionality, it is recommended to test one-by one using the modified code referring to Nuvoton board support package on GitHub.
Link: (https://github.com/OpenNuvoton/m460bsp/tree/master/SampleCode/StdDriver)

- GPIO



1. Make an LED circuit with a breadboard. Use random GPIO pins to test it with a basic LED blink program.
2. Set the desired port as output first with
   GPIO_SetMode([port used], [bit of the port], GPIO_MODE_OUTPUT);
   For example, if we want to use PG.2, write code in int main(void){}
   GPIO_SetMode (PG, BIT2, GPIO_MODE_OUTPUT);

- Relay
  1. Set the Relay GPIO (PD0-5) as output. For example, to set R.STO pin as output write GPIO_SetMode(PD, BIT0, GPIO_MODE_OUTPUT);
  2. Set logic 1 to the relay GPIO pins (PD0-5) to activate the relay. For example, if we want to test R.STO, write PD0=1;
  3. If the relay is activated, there will be a clicking sound from relay switching.
- Clock
  Use CLK_ClockDetector code from Nuvoton M460 BSP to test the HXT and LXT
- ADC
  1. Set the ADC pin to PB.0 for EADC0_CH0 and PB.1 for EADC0_CH1



  2. Use the average-mode BSP sample code for testing. In this program, one sample module is configured to measure the selected channel, and the ADC performs 32 accumulated conversions with average enabled before the result output. Each ADC channel is tested individually by selecting the option in the terminal: press ‘1’ for channel 0 and ‘2’ for channel 1. The averaged conversion result is then printed through UART0 to the serial terminal.
  3. Adjust the potentiometer to vary the analogue input voltage and observe the corresponding averaged ADC result from the serial terminal output. A regulated power supply may also be used instead of the potentiometer when a precise correlation between input voltage and digital conversion value is required.
- DAC

1. In int main(void){}, set DAC0 and DAC1 voltage output at PB.12 and PB.13, respectively.
2. Use the group-mode code from BSP. The program uses a sine lookup table and interrupt-driven updates, so both DAC channels convert simultaneously, with DAC1 intentionally shifted by 180° relative to DAC0.
3. Connect an oscilloscope probe to DAC test points and verify that both channels output stable sine waves with matching amplitude and frequency, and that the two waveforms are phase-opposite (peaks of one align with troughs of the other)

- PWM
  1. Set BPWM pins used in sys_init(void){}. The syntax is SET_BPWMX_CHY_pin()
     X=0/1, depends on chosen BPWM
     Y=Channel of BPWM0/BPWM1
     pin=Selected BPWM pins. For one channel of BPWM0 or BPWM1, there are several pin options.
     Example: SET_BPWM1_CH0_PB11();
  2. In main(void){}, configure the BPWM channel, period (in ms) and duty cycle inside BPWM_ConfigOutputChannel() function.
     Example: BPWM_ConfigOutputChannel(BPWM1, 0, 60000, 50);
  3. Enable the PWM output with BPWM_EnableOutput()
     Usage: BPWM_EnableOutput(BPWM_T *BPWM, uint32_t u32channelmask);
     BPWM_T *BPWM=BPWM0/BPWM1
     uint32_t u32channelmask=bit mask (in hex) to assign starting channel(s). e.g.: 0x10 channel 0 only, 0x3f all channels simultaneously.
     Example: BPWM_EnableOutput(BPWM0, 0x3F);
  4. Using an oscilloscope, observe the waveform of PWM signals from the test points (TP11 for BPWM0, TP12 for BPWM1), shifted from 3V3 (original PWM output from MCU) to 5V with a logic level shifter.
  5. Stop the PWM wave (optional) with BPWM_ForceStop()
- UART
  1. Use USB to TTL cable as NuLink-Pro does not have VCOM feature. Connect the RX cable to the TX pin of the desired UART channel, and the TX cable to the RX pin.
  2. Can be tested simultaneously with ADC or use the template code from Nuvoton BSP.
  3. Observe the output on serial with baud rate 115200.
- CAN
  1. Put MCP2551 module on J23 and J24 each
  2. Program the MCU with the modified CAN BSP test firmware.
  3. Open a serial terminal through UART0 at 115200 baud.
  4. Run the test and observe the UART output.
  5. Confirm that:
     a. CAN0 transmits ID 0x123 and CAN1 receives it correctly
     b. CAN1 transmits ID 0x321 and CAN0 receives it correctly
     c. Both results show **PASS** in the terminal output.
- Ethernet

1. Connect the PCB to a laptop or PC using an RJ45 Ethernet cable.
2. Power the PCB and check whether the Ethernet port LEDs turn on or blink, indicating possible physical link activity.
3. Load and run the Ethernet test firmware based on the Nuvoton BSP EMAC example. Before building the program, make sure the required EMAC driver files and related libraries are added to the project correctly.
4. Open the UART terminal to monitor the program output and verify whether the firmware reaches EMAC initialization and attempts to obtain an IP address through DHCP.
5. If a valid Ethernet link is established and the board successfully obtains an IP address, test the communication by sending a ping command from the PC to the assigned IP address.
6. If the ping fails and no Ethernet port LED is active, further inspect the physical Ethernet layer, including cable condition, RJ45/magnetics connection, PHY initialization, and clock/reset signals.